

动态规划选讲

Larunatrecy

- #### ④ 数据结构优化 dp

$$\forall l_1 \leq l_2 \leq r_1 \leq r_2, w(l_1, r_1) + w(l_2, r_2) \leq w(l_1, r_2) + w(l_2, r_1)$$

定理

证明.

$w(g(i), j) \geq w(g(j), j), w(g(j), i) > w(g(i), i)$, 相加得
 $w(g(i), j) + w(g(j), i) > w(g(j), j) + w(g(i), i)$ 与四边形不等式矛盾。

那么如果我们现在知道了 w 满足四边形不等式，并且可以快速计算，该如何求解原问题呢？
接下来介绍两种做法。

均摊下来每次只会加入一个段，每次遍历的段都删了，加上二分时间复杂度为 $O(Tn \log n)$ ，其中 T 为单次计算 w 的复杂度。

比较二分队列和分治算法的优劣

二分队列

- 优势：可以计算形如 $f_i = \min_{j \leq i} f_{j-1} + w(j, i)$ 的半在线问题。
- 劣势：需要 **完全单调性**，贡献 w 需要能快速独立计算。

分治算法

- 优势：可以用双指针维护同层的 w 快速转移。
- 劣势：无法直接解决半在线的决策单调性问题。

再看半在线决策单调性

我们现在还是要计算 $f_i = \min_{j \leq i} f_{j-1} + w(j, i)$ ，并且 w 是类似逆序对这样无法快速单独计算，但是可以快速维护增量的函数，且 w 满足完全单调性。考虑在决策单调性分治的基础上，套一层 CDQ 分治，算法流程如下：

设 $\text{CDQ}(l, r)$ 为求出 $f_l \sim f_r$ 。

- 如果 $l = r$ 直接返回。
- 否则, 设 $mid = (l + r) / 2$, 先递归 $CDQ(l, r, mid)$, 然后调用决策单调性分治函数 $solve(mid + 1, r, l, mid)$, 然后递归 $CDQ(mid + 1, r)$

时间复杂度 $O(Tn \log^2 n)$, 其中 T 为单次计算 w 增量的复杂度。
能不能做的更快啊!

再看半在线决策单调性

设 $\text{solve}(l, r, L, R)$ 为已经求出 $f_1 \sim f_{l-1}$, l 的转移点是 L , r 在 $1 \sim l-1$ 内的最优转移点是 R , 现在要求出 $f_l \sim f_r$, 算法流程如下:

- 首先, 用 $L \sim R$ 内的转移点去更新 mid , 设最优转移点为 p 。
- 此时, $[l, mid]$ 已经符合递归条件, 可以递归执行 `solve(l,mid,L,p)`。
- 然后用 $l \sim mid$ 内的转移点去更新 r 的转移点, 之后可以发现符合 $[mid, r]$ 的转移条件了, 递归执行 `solve(mid+1,r,p,R)`。

时间复杂度 $O(Tn \log n)$, 其中 T 为单次计算 w 增量的复杂度。

[USACO19FEB] Mowing Mischief P

假设没有横纵坐标都比 i 小的限制，如何快速地转移，考虑决策单调性。设上一层的两个决策点 $j < k$ 。如果 k 比 j 更优：

$$\begin{aligned} dp_k + (x_i - x_k)(y_i - y_k) &\leq dp_j + (x_i - x_j)(y_i - y_j) \\ x_i(y_j - y_k) + y_i(x_j - x_k) &\leq (dp_j + x_j y_j) - (dp_k + x_k y_k) \end{aligned}$$

所以 i 越大，上面的不等式越不容易满足，所以决策点是随着 i 的变大而变小的。

考虑区间的限制，可以使用线段树分治，把询问点按照可行的决策点区间放入 $\log$ 个线段树节点，然后再在每个线段树的节点内跑决策单调性。

时间 $\mathcal{O}(n \log^2 n)$ 。

CF2061I Kevin and Nivek

设 $f_{i,j}$ 为前 i 场比赛, A 比 B 多赢 j 场的最小代价。

考虑如果 $|j| \geq B$, 那么从 i 开始往后的 B 场比赛内第二种要么都选要么都不选, 而剩下的肯定是按照 a_i 从小到大排序后选一个前缀, 和上个题一样具有决策单调性, 因此可以直接从 f_i 整体转移到 f_{i+B} 。

对于 $|j| \leq B$ 暴力 dp , 取 $B = \sqrt{n \log n}$, 时间复杂度 $O(n\sqrt{n \log n})$ 。

CF2061I Kevin and Nivek

改成分治, 设 $\text{solve}(l, r)$ 为从 $f_{l,j} (|j| \leq r - l + 1)$ 转移到 $f_{r,k}$ 。则对于 $|j| > mid - l + 1$ 可以直接决策单调性转移到 f_{mid} , $|j| \leq mid - l + 1$ 的递归分治即可。
时间复杂度 $O(n \log^2 n)$ 。

[UNR 5] 航天飞机调度

题目描述

给定一个 n 个点的三角剖分，有两架飞机初始时分别在 x, y 。
有 q 次调度需求，第 i 次需要选择一架飞机移动到 p_i ，代价是这两点间的最短路。

求出总调度价值的最大值。

图是未知的，你可以多次询问两点间最短路，询问次数 $O(q \log n)$ 。

[UNR 5] 航天飞机调度

设 $f_{i,j}$ 为第 i 次调度式，一架飞机在 p_i 一架在 j 的最大值，转移：

- $f_{i+1,j} \leftarrow f_{i,j} + \text{dis}(p_i, p_{i+1})$
- $f_{i+1,p_i} \leftarrow f_{i,j} + \text{dis}(j, p_{i+1})$

将点按照换上的位置编号并复制到 $[n+1, 2n]$ ，则满足如下性质

$$\forall u \leq v \leq x \leq y < u + n$$

$$\text{dis}(u, x) + \text{dis}(v, y) \geq \text{dis}(u, y) + \text{dis}(v, x)$$

考虑设置矩阵 $W_{x,y}$ 为：

- 若 $y \in [x, x+n-1]$ ，则 $W_{x,y} = \text{dis}(x, y)$ 。
- 否则 $W_{x,y} = -\infty$

则 W 满足四边形不等式。

- ## ④ 数据结构优化 dp

凸函数

设 $f(x)$ 为定义域在非负整数的函数, 若

$\forall x, f(x+1) - f(x) \geq f(x) - f(x-1)$, 则称 $f(x)$ 为一个下凸函数, 相应的定义上凸函数。

wqs 二分

wqs 二分，又称为带权二分，凸优化，是一种处理形如 **有若干个物品，求选择恰好 k 个物品的最大/最小价值** 的问题的常用手段。具体的，如果 $f(x)$ 表示选择了恰好 x 个物品的最值，并且 $f(x)$ 是一个下凸函数，我们希望求出 $f(k)$ 的值。

wqs 二分

考虑用一条斜率为 K 的直线去切这个凸壳并求出切点，那么根据斜率优化的相关结论当截距最小时是切点，即找一个 x_0 ，最小化 $f(x_0) - Kx_0$ 的值，此时没有恰好 k 个的限制，设

$g(x) = f(x) - Kx$ ，求解 $g(x)$ 的最优值即可。

回到原问题，因为 f 是下凸函数，所以随着 K 的增大，切点肯定也增大。那么我们二分斜率 K ，如果大于 k 就调小 K 否则就调大，这样最终我们就能求出 $x = k$ 时的切线斜率。

对于 $g(x)$ ，相当于我们每选一个物品就会附加 $-K$ 的惩罚，这样如果惩罚大了物品肯定就选的少，我们就可以通过调整惩罚值不断逼近目标物品数量 k 。

例题

[国家集训队] Tree I

给你一个无向带权连通图，每条边是黑色或白色。让你求一棵最小权的恰好有 k 条白色边的生成树。

种树

给定一个序列 a ，求序列里选出 k 个数，要求选出的数不相邻，求最大权值，

[IOI 2000] 邮局加强版加强版

高速公路旁边有 n 个村庄。高速公路表示为整数轴，每个村庄的位置用单个整数坐标标识。两个位置之间的距离是其整数坐标差的绝对值。

现在要建立 m 个邮局，邮局将建在一些，但不一定是所有的村庄中。为了建立邮局，应选择他们建造的位置，使每个村庄与其最近的邮局之间的距离总和最小。

P10181 龙逐千灯幻

题目描述

定义 $f(A, k)$ 为把序列 A 划分成 k 段，每段的颜色个数之和的最大值。

给定长度为 n 的序列 $A_1 \sim A_n$ ， q 次询问，每次给出 i, k ，你要求出 $f(A_{1 \dots i}, k)$ 。

$1 \leq n \leq 10^5, 1 \leq q \leq 10^6$

P10181 龙逐千灯幻

考虑能不能把线段树的 $\log$ 去掉。

观察一下我们实际需要的操作：

- 向末尾加入一个数
- 后缀加 1
- 求最大值

维护一个单调递减的单调栈，那么 1 操作可以直接不断弹栈维护，3 操作就是查询栈底元素。

对于 2 操作，我们找到该后缀在单调栈上对应的位置，这个可以用并查集维护，然后相当于把这个部分往前面合并弹出若干元素，最后打上 +1 标记。

因为要支持中间弹出元素，所以我们用链表维护这个单调栈，至于 +1 标记，我们可以发现我们相当于只需要查询栈底，查询栈顶，查询某相邻两个位置的差值，因此直接维护单调栈内元素的差分即可，打标记是简单的。

瓶颈在于并查集，因此单次 dp 的复杂度优化到了 $O(n\alpha(n))$ ，可以通过此题。

CF2183H Minimise Cost

题目描述

$$f(b) = m \times \sum_{i=1}^m b_i$$

给定一个序列 a ，要求将其划分为 k 个不交的非空子序列 $s_1, s_2, \dots, s_k$ ，最小化 $\sum f(s_i)$ 。

$n \leq 2 \times 10^5$ 。

CF2183H Minimise Cost

题目描述

$$f(b) = m \times \sum_{i=1}^m b_i$$

给定一个序列 a ，要求将其划分为 k 个不交的非空子序列 $s_1, s_2, \dots, s_k$ ，最小化 $\sum f(s_i)$ 。

$n \leq 2 \times 10^5$ 。

ZROI2844 木芙蓉

题目描述

有一棵 n 个节点的树，边上有边权，点上有点权。

我们需要做恰好 k 次选择。每次选择一个节点和他的两条邻边，这里要求每条边只能被选择一次。

可以获得的收益是所有被选择了至少 1 次的节点的点权之和减去所有被选择的边的边权和。

请求出收益最大是多少，并构造方案。

$n \leq 2 \times 10^5$ 。

ZROI2844 木芙蓉

显然外面要 wqs 二分，难点在于输出方案。不难发现，wqs 二分中切到最小值的是一段区间 $[l_i, r_i]$ ，因此在转移的时候记录最小值和最大值，输出方案的时候，如果 k 位于合法区间内就转移即可。

不可视境界线 [环版本]

首先，假如我们能任意固定起点跑出一组决策单调性的方案，那么与之前的环形决策单调性类似，所有段的决策点是单调的，还是对最小段作为起点分治，可以做到 $O(n \log^2 n)$ 。

求方案

dp 套 dp

我们经常遇到对满足某种条件的结构计数的问题，但有时这种结构并不能直观的观察出来是否满足，此时可以考虑设计另一个 dp 来判定这种条件是否满足，然后把 dp 数组的值状压下来去进行外层的计数。

通常的写法是，爆搜所有可能的状态，对于每个状态向他下一步可能转移到的状态连边，形成一个确定有限状态自动机（DFA）。然后外层的转移就是在 DFA 上移动的过程，通常情况下这个 DFA 的状态数不会很大。

[TJOI2018] 游园会

题目描述

令 $f(i)$ 为满足下面几条限制的字符串个数：

- 长度为 n ，字符集为 $\{N, O, I\}$ 。
- 不包含子串 NOI。
- 与模式串 s 的最长公共子序列 (lcs) 长度为 i 。

对于 $i \in [0, k]$ ，求出 f_i 的值，对 $10^9 + 7$ 取模。

[SDOI/SXOI2022] 小 N 的独立集

题目描述

给定 n 个点的树的形态以及点的权值范围 k ，对于任意 $i \in [1, kn]$ ，求有多少种权值分配方案，使得树的最大权独立集大小为 i 。

$n \leq 1000, k \leq 5$ 。

[SDOI/SXOI2022] 小 N 的独立集

容易想到设计 $f(u, x, y)$ 表示给以 u 为根的子树填数，选 u 时的最大权独立集大小为 x ，不选 u 时为 y 的方案数。转移就直接写一下树上最大权独立集那个 DP，有

$$f(u, x_1, y_1) \times f(v, x_2, y_2) \rightarrow f(u, x_1 + y_2, y_1 + \max(x_2, y_2))$$

复杂度是 $O(n^3 k^4)$ 。

然后发现，当 $x \leq y$ 的时候记录 x 没有意义，若 $x > y$ ，那么 $x - y \leq k$ 。因此可以记录 $f(u, j, y)$ 表示以 u 为根的子树填数，选 u 时的最大权独立集大小为 $y + j$ ($j > 0$) 或者 $\leq j$ ($j = 0$)，不选 u 时最大权独立集大小为 y 的方案数。转移式类似：

$$f(u, j_1, y_1) \times f(v, j_2, y_2) \rightarrow f(u, \max(0, j_1 - j_2), y_1 + y_2 + j_2)$$

复杂度 $O(n^2 k^4)$ 。

如何做的更好？

[PKUSC2024] 独立

题目描述

给定 n 个点的树的形态以及点的权值范围 m ，求出最大权独立集的期望。

$n \leq 2000, m \leq 10^8$ 。

[PKUSC2024] 独立

考虑求最大权独立集的 dp 算法： $dp_{u,0/1}$ 表示 u 没选 / 选时的最大独立集，转移如下：

$$dp_{u,0} = \sum_{v \in \text{son}(u)} \max(dp_{v,0}, dp_{v,1})$$

$$dp_{u,1} = w_u + \sum_{v \in \text{son}(u)} dp_{v,0}$$

其中 w 表示权值，记 $a_u = \max(dp_{u,0}, dp_{u,1}) - dp_{u,0}$ ，带入可得 $a_u = \max(0, w_u - \sum_{v \in \text{son}(u)} a_v)$ 。

并且整棵树的最大权独立集就等于 $\sum a_u$ ，因此可以根据 a_u 设计 dp 套 dp 算法。

设 $f_{u,i}$ 表示 $a_u = i$ 的方案数，那么转移时先求 g 为 f_v 的卷积，然后 $f_{u,\max(j-i,0)} \leftarrow g_i$ ，其中 $j \in [1, m]$ 表示 w_u 。

答案就是 $\sum_u \sum_i m^{n-\text{size}(u)} \times i \times f_{u,i}$ 。

[PKUSC2024] 独立

注意到我们要对 $f_{u,0}$ 特殊处理，用 $m^{\text{size}(u)} - \sum_{i>0} f_{u,i}$ 更新，此时 $f_{u,i} = \sum_{j=1}^{m-i} g_j$ 。

可以用归纳法证明，对于每个 u ， $f_{u,1} \sim f_{u,m}$ 是某个 $\mathcal{O}(n)$ 次多项式在 $x = 1 \sim m$ 处的点值。

不妨设 $k = n + \mathcal{O}(1)$ 表示多项式最高次数，我们只要维护 $f_{u,1} \sim f_{u,k}$ 即可插值出整个 f_u 。

考虑如何转移，首先我们发现 $f_{u,m-k} \sim f_{u,m}$ 是容易求的，只要求 $g_0 \sim g_k$ 即可，这个可以用 NTT 快速求出。

然后我们要考虑 $f_{u,m-k} \sim f_{u,m}$ 通过拉格朗日插值求出 $f_{u,1} \sim f_{u,k}$ ，这个也可以求出式子后用 NTT 加速。

最后我们要求 $\sum_{i=1}^m f_{u,i}$ 来更新 $f_{u,0}$ ，求 $\sum_{i=1}^m i \times f_{u,i}$ 来更新答案，显然这两个值也是关于 m 的 $\mathcal{O}(n)$ 次多项式，通过

$f_{u,1} \sim f_{u,k}$ 来插值求答案即可。

时间复杂度 $\mathcal{O}(n^2 \log n)$ 。

1. *Journal of Management Studies*, 1990, 27, 1, 1-14.

1. *Journal of Management Studies*, 1995, 32, 1, 1-14.

一列一列考虑，上一列的每个格子有以下五种状态：

- 月 01 表示状态合不合法, 共有 5^3 个状态。

那么计算填数方案时，只需要 2^{5^3} 把每种状态是否合法都压下来即可。爆搜状态数和转移矩阵，然后矩阵快速幂即可。

[NOI2022] 移除石子

考虑如何判断一个 $\{a_n\}$ 能否被移除。

简化题意，先假设 $k = 0$ ：设计 $f_{i,j,k}$ 表示考虑 $1 \sim i$ ，钦定 j 个二操作至少延伸到 i ， k 个操作至少延伸到 $i + 1$ ，是否可行。转移要枚举以这一位为终点和起点的二操作分别有多少个，设有 l 个以 i 为起点，则要求 $a_i - j - k - l \in \{x | x = 0 \vee x \geq 2\}$ 。需要优化。

注意到:

- 不可能出现相同的二操作，否则可以用一操作代替；
- 操作区间长度只能为 $\{3, 4, 5\}$ 否则可以分裂为更小的区间的共同效果。
- 每个点出发的区间数小于等于 3，即 $l \leq 3$ ，所以 $k \leq 3$ ；
- 到 i 时长度已达 3 的区间只有：起点 $i - 2$ ，长度为 3, 4, 5；起点 $i - 3$ ，长度为 4, 5；起点 $i - 4$ ，长度为 5；共 6 种，即 $j \leq 6$ 。

进一步的优化发现：同时出现 4,5 是可以用一次一操作和 3,4 代替，所以 $j \leq 4, k \leq 2$ 。

动态 dp

有一个长度为 n 的序列 a_i ，每个数可以选或者不选，但相邻两个数不能同时选，最大化选出的数的和。

有一个简单的 dp ，设 $f_{i,0/1}$ 表示前 i 个数，第 i 个数是否选了的
最大价值，转移时

- $f_{i,1} = f_{i-1,0} + a_i$
- $f_{i,0} = \max(f_{i-1,0}, f_{i-1,1})$

现在把这个问题加强一下：

有一个长度为 n 的序列 a_i ，每个数可以选或者不选，但相邻两个数不能同时选，最大化选出的数的和，需要支持单点修改和对区间查询。

动态 dp

我们可以把转移写成矩阵的形式：

$$\begin{bmatrix} f_{i,0} & f_{i,1} \end{bmatrix} = \begin{bmatrix} f_{i-1,0} & f_{i-1,1} \end{bmatrix} \times \begin{bmatrix} 0 & a_i \\ 0 & -\infty \end{bmatrix}$$

其中矩阵乘法定义为 $(\max, +)$ 乘法。

不妨令

$$M_i = \begin{bmatrix} 0 & a_i \\ 0 & -\infty \end{bmatrix}$$

那么答案可以写成

$$\begin{bmatrix} 0 & 0 \end{bmatrix} \times \prod_{i=1}^n M_i$$

现在是单点修改，区间查询，我们可以用线段树维护区间内的 M_i 的乘积，因为矩阵乘法有结合律所以这样做是正确的。这样子就可以在 $\Theta(n \log n)$ 复杂度内解决这个问题。

多次询问一个字符串的一个子串至少删去多少个字符后才能使得其不包含子序列 2016 而包含 2017。

树上的情况（静态）

对于静态情况，我们可以使用树上倍增，预处理 $f_{i,k}$ 表示从 i 到 i 的 $2^k - 1$ 级祖先的这条链上的矩阵的乘积，这样我们就可以快速查询出一条链的矩阵乘积了。

P5024 [NOIP2018 提高组] 保卫王国

P8820 [CSP-S 2022] 数据传输

全局平衡二叉树/静态 Toptree

每次树剖选取带权中点建线段树，权值为区间内的轻子树大小之和。轻子树类似的按照大小排序后每次选取带权中点划分，类似的建立二叉树。从而可以把整棵树变成一棵自底向上更新的、高度为 $O(\log n)$ 的二叉树。

[ZJOI2022] 深搜

题目描述

定义 $f(x, y)$ 合法当且仅当 y 在 x 子树中，其值为对 x 的子树进行 dfs，往下走随机选一个没有访问过的点，遇到 y 时经过的点权最小值的期望。

求 $\sum_{x,y} f(x,y)$ 。
$$1 \leq n \leq 4 \times 10^5.$$

[ZJOI2022] 深搜

拆贡献，考虑算 $\min = w$ 的点对个数，然后转化成 $\min \geq w$ 的点对个数。

我们将 $\min \geq w$ 的点称为好点，否则称为坏点。如果一颗子树内的点都是好点，则成为好子树，否则称为坏子树。

考虑 $\text{dfs}(x,y)$ 的进程, 假设现在在 $x \rightarrow y$ 的路径上点 p :

如果随机到的儿子子树内含 y , 合法;

随机到一颗好子树，合法；

随机到一颗不含 y 的坏子树，不合法。

考虑不出现不合法情况的概率，注意到好子树没用，相当于在坏子树和含 y 子树中随机第一个是含 y 子树，换言之记坏子树个

数为 c ，则概率为 $\frac{1}{c+1}$ 。

[WC 2024] 水鏡

题目描述

给定一个长度为 n 的正整数序列 $h_1, h_2, \dots, h_n$ ，求满足以下所有条件的二元组 (u, v) 的数量：

- $1 \leq u < v \leq n$, 且 u, v 为整数;
- 存在一个 ** 正实数 ** L 以及一个长度为 $(v - u + 1)$ 的序列 $r_u, r_{u+1}, \dots, r_v$ 满足以下所有条件:
- $\forall u \leq i \leq v$, 记 $h'_i = 2L - h_i$, 则 $r_i \in \{h_i, h'_i\}$, 特别地, 当 $h_i = h'_i$ 时, $r_i = h_i$;
- $\forall u \leq i < v, r_i < r_{i+1}$.

$$2 \leq n \leq 5 \times 10^5, \quad \forall 1 \leq i \leq n, 1 \leq h_i \leq 10^{12}.$$

[WC 2024] 水鏡

令 $T = 2L$ 。

可以直观感受到 T 分成 $O(n)$ 个有效段，因为我们只关心相邻两个数关于 T 的相对距离，所以界点会是 $2a_i$ 或者 $a_i + a_{i+1}$ 。

有 $O(n^2)$ 暴力, 设 $f(t, i, 0/1)$ 表示 $T = t$ 时, 不反转/反转 i 为右端点时拓展的最大距离。

1. $a_i < a_{i+1}, f(t, i, 0) \rightarrow f(t, i + 1, 0)$
- 2.

$$a_i > a_{i+1}, f(t, i, 1) \rightarrow f(t, i + 1, 1) \text{ 3.}$$
$$a_i + a_{i+1} < T, f(t, i, 0) \rightarrow f(t, i + 1, 1) \quad 4.$$
$$a_i + a_{i+1} > T, f(t, i, 0) \rightarrow f(t, i + 1, 1)$$

前面两种转移是固定的，后面两种转移关于 T 只最多分成三段。

转移可以用 $(max, +)$ 矩乘表示, 考虑按照下标做扫描线, 线段树维护每一项 T 的 f 矩阵。转移时区间乘矩阵就好了。

复杂度 $O(n \log n)$ 。

因此原问题转化为 A 性质，即一条链。

夏終

考虑最优解选择的 E 中的边为集合 P , 那么这些边将图连成一个联通块, 选择 $x \in \operatorname{argmin}\{b_i\}$, E' 中边的最优选择方法显然为不包含 x 的联通块的选择一个点和 x 连边。

令 $C' = b_x + C$ ，相当于除了连的边外，一个联通块 S 还需要额外付出 $\min_{i \in S} \{b_i\} + C'$ 的代价（最后答案需要减去 $b_x + C'$ ）。

考虑 DP, 记 w_i 为连接 $i-1$ 和 i 的边的边权, 设 $f_{i,j,0/1}$ 为考虑 $[1,i]$, 有 j 个联通块, 最后一个联通块仍未计算/已经计算额外代价的最小代价和。转移:

$$f_{i,j,0} = \min(f_{i-1,j-1,1}, f_{i-1,j,0} + w_i)$$

$$f_{i,j,1} = \min(f_{i-1,j,1} + w_i, f_{i-1,j-1,1} + b, f_{i-1,j,0} + w_i + b_i)$$

答案即为 $\min\{f_{n,i,1} + i \times C'\}$.

给定 n 个点的一棵树，每个点有权值 w_i 。
有 q 次询问，每次给定 l, r ，你需要求出仅保留树上编号在 l, r 内的点形成的森林的最大权独立集的值。
对于 100% 的数据， $1 \leq n, q \leq 10^5, 0 \leq w_i \leq 10^5, 1 \leq l \leq r \leq n$

有 q 次询问，每次给定 l, r ，你要求出仅保留树上编号在 l, r 内的点形成的森林的最大权独立集的值。

对于 100% 的数据, $1 \leq n, q \leq 10^5, 0 \leq w_i \leq 10^5, 1 \leq l \leq r \leq n$

下弦月

考虑扫描线，建出静态 Top tree 来维护我们的扫描线，在扫描线的过程中对于每个点数小于 $\sqrt{n}$ 的簇维护一个大小为簇大小的数组 f_i 表示只考虑编号大于 i 的点时这个簇的独立集信息（钦定上下界点选或者不选时的最大独立集大小），当你新加入一个点时就从下往上更新 f 数组，这里复杂度是一个等比数列求和，容易发现是 $O(\sqrt{n})$ 的，当你询问时就对于 Top tree 上点数大于等于 $\sqrt{n}$ 的簇整体重新做一次 dp 即可，复杂度也是 $O(\sqrt{n})$ 的，在簇内离散化编号可以离线双指针，把空间压到线性可以离线 dfs 每个簇处理信息以及信息合并，至此本题在 $O((n+q)\sqrt{n})$ 时间 $O(n+q)$ 空间内解决。

CF1740H MEX Tree Manipulation

提供一个 $O(n \log n)$ 的做法。

首先一个点的权值是 $O(\log n)$ 级别的，这是因为设 f_k 表示权值为 k 的树的最小大小，那么有 $f_k = \sum_{i=0}^{k-1} f_i + 1$ ，解得 $f_k = 2^k$ 。那么只需要用一个数状压下儿子中存不存在权值为 k 的点，每次 lowbit 一下，便可做到 $O(1)$ 动态维护一个点的 MEX 值。

接下来就是怎么维护权值了，考虑 DDP，树剖之后假设一个点所有轻儿子的权值都知道，那么该点权值关于该点重儿子的权值的函数 $g(x)$ 可以快速复合。进一步地，我们可以在复合的时候维护出复合的前缀和函数，大概是

$h(x) = g_1(x) + g_2(g_1(x)) + g_3(g_2(g_1(x))) \dots$ 这种形式。

于是使用全局平衡二叉树维护信息，每次采用上面状压的技巧更新轻儿子。时间复杂度 $O(n \log n)$ ，目前是 CF 最快解。

为了实现方便，代码中的 MEX 值实际上是 +1 了的。

Thanks!